

Part 2: Common Stream Operations

In this part, you'll learn the Stream operations that you'll use almost daily in Java backend development.

Topics covered:

1.  `distinct()`
 2.  `limit()`
 3.  `skip()`
 4.  `count()`
 5.  `collect()` → List, Set, Map
 6.  `groupingBy()` for report-style aggregation
-

1. `distinct()` – Remove Duplicate Elements

What is `distinct()`?

`distinct()` removes duplicate elements from a stream.

It uses the object's **`equals()`** and **`hashCode()`** methods to determine whether two elements are equal.

Returns: A new stream with unique elements.

Example 1: Remove Duplicate Integers

```
List nums = List.of(1, 2, 2, 3, 4, 4, 5);

nums.stream()
    .distinct()
    .forEach(System.out::println);
```

Output

```
1
2
```

```
3  
4  
5
```

Example 2: Remove Duplicate Strings

```
List names = List.of(  
    "Ali",  
    "Sara",  
    "Ali",  
    "Ahmed",  
    "Sara"  
);  
  
names.stream()  
    .distinct()  
    .forEach(System.out::println);
```

Output

```
Ali  
Sara  
Ahmed
```

Example 3: With Custom Objects

```
class Student {  
    int id;  
    String name;  
  
    Student(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

```
students.stream()  
    .distinct()  
    .forEach(System.out::println);
```

If equals() and hashCode() are **not overridden**, duplicates **won't be removed**.

Real Project Example

```
emails.stream()  
    .distinct()  
    .forEach(System.out::println);
```

Useful when sending one email per unique address.

Common Mistake

Thinking distinct() works on custom objects automatically.

 Wrong

Override equals() and hashCode().

Interview Questions

Q1. How does distinct() identify duplicates?

Answer

Using equals() and hashCode().

Q2. Does distinct() modify the original list?

No.

One-liner

distinct() removes duplicate elements using equals() and hashCode().

2. limit() – Take First N Elements

What is limit()?

Returns only the first **N** elements.

Signature

```
limit(long maxSize)
```

Example 1

```
List nums =  
    List.of(1,2,3,4,5,6);  
  
nums.stream()  
    .limit(3)  
    .forEach(System.out::println);
```

Output

```
1  
2  
3
```

Example 2

```
List names =  
    List.of("Ali", "Sara", "Ahmed", "John");
```

```
names.stream()  
    .limit(2)  
    .forEach(System.out::println);
```

Output

```
Ali  
Sara
```

Real Project Example

Show only the latest five notifications.

```
notifications.stream()  
    .limit(5)  
    .forEach(System.out::println);
```

Interview Question

Does `limit()` change the original collection?

No.

One-liner

`limit()` returns only the first specified number of elements.

3. `skip()` – Ignore First N Elements

What is `skip()`?

Skips the first **N** elements.

Example 1

```
List nums =  
    List.of(1,2,3,4,5);  
  
nums.stream()  
    .skip(2)  
    .forEach(System.out::println);
```

Output

```
3  
4  
5
```

Example 2

Pagination

```
products.stream()  
    .skip(20)  
    .limit(10)  
    .forEach(System.out::println);
```

This returns page 3 (assuming 10 items per page).

Real Project Example

Pagination in REST APIs.

Common Mistake

Order matters.

Correct

```
stream.skip(5).limit(10);
```

Different result

```
stream.limit(10).skip(5);
```

Interview Question

Can `skip()` be used with `limit()`?

Yes.

Very common for pagination.

One-liner

`skip()` ignores the first N elements.

4. `count()` – Count Elements

Returns the number of elements.

Returns

```
long
```

Example 1

```
List nums =  
    List.of(2,4,6,8);  
  
long total =  
    nums.stream()  
        .count();  
  
System.out.println(total);
```

Output

4

Example 2

Count Adults

```
long adults =  
ages.stream()  
    .filter(age -> age >= 18)  
    .count();
```

Output

3

Real Project Example

```
long activeUsers =  
users.stream()  
    .filter(User::isActive)  
    .count();
```

Common Mistake

Thinking

```
count()
```

returns int.

It returns long.

Interview Questions

Is `count()` intermediate?

No.

It is a **terminal operation**.

One-liner

```
count() returns the total number of stream elements as a long.
```

5. `collect()` – Create List, Set, and Map

One of the most important Stream operations.

`collect()` gathers stream results into another collection or data structure.

Collect into List

```
List result =  
names.stream()  
    .filter(n -> n.startsWith("A"))  
    .collect(Collectors.toList());
```

Example

```
List even =  
nums.stream()  
    .filter(n -> n % 2 == 0)  
    .collect(Collectors.toList());  
  
System.out.println(even);
```

Output

```
[2,4,6]
```

Collect into Set

```
Set unique =  
nums.stream()  
    .collect(Collectors.toSet());
```

Duplicates disappear.

Example

```
List nums =  
List.of(1,2,2,3);  
  
Set unique =  
nums.stream()  
    .collect(Collectors.toSet());  
  
System.out.println(unique);
```

Output

```
[1,2,3]
```

Collect into Map

Suppose

```
class Employee{  
    int id;  
    String name;  
}
```

Convert to

```
id → employee
```

```
Map map =  
employees.stream()  
    .collect(  
        Collectors.toMap(  
            Employee::getId,  
            e -> e  
        )  
    );
```

Another Example

```
Map lengths =  
names.stream()  
    .collect(  
        Collectors.toMap(  
            name -> name.length(),  
            name -> name  
        )  
    );
```

```
        name -> name,  
        String::length  
    )  
);
```

Output

```
Ali = 3  
  
Ahmed = 5
```

Real Project Example

Convert List of Users into a Map for quick lookup.

Common Mistake

Duplicate keys

```
Collectors.toMap(...)
```

throws

```
IllegalStateException
```

when duplicate keys exist.

Correct

```
Collectors.toMap(  
    Employee::getId,  
    e -> e,
```

```
(oldValue,newValue) -> oldValue  
)
```

This merge function resolves duplicate keys.

Interview Questions

Difference between `toList()` and `toSet()`?

List keeps duplicates.

Set removes duplicates.

Can `toMap()` fail?

Yes.

Duplicate keys cause an exception unless a merge function is provided.

One-liner

`collect()` gathers stream elements into collections such as List, Set, or Map.

6. `groupingBy()` – Report Style Aggregation

One of the most common interview questions.

Groups elements based on a key.

Think SQL

```
GROUP BY
```

Suppose

```
Employee  
  
name  
  
department
```

Example 1

```
Map> map =  
employees.stream()  
    .collect(  
        Collectors.groupingBy(  
            Employee::getDepartment  
        )  
    );
```

Result

```
IT  
  
Ali  
  
Ahmed  
  
HR  
  
Sara  
  
John
```

Example 2

Group Numbers

```
Map> result =  
  nums.stream()  
    .collect(  
      Collectors.groupingBy(  
        n -> n % 2 == 0 ? "Even": "Odd"  
      )  
    );
```

Output

Even

2

4

6

Odd

1

3

5

Example 3

Group by String Length

```
Map> result =  
  names.stream()  
    .collect(  
      Collectors.groupingBy(  
        String::length  
      )  
    );
```

Output

```
3
Ali
Bob
5
Ahmed
```

Real Project Example

Group employees by department.

Group products by category.

Group orders by customer.

Common Mistake

Thinking `groupBy()` returns a `List`.

It returns a

```
Map>
```

Interview Questions

Which collection does `groupBy` return?

A `Map`.

Is groupingBy terminal?

Yes.

It is used inside collect().

One-liner

groupingBy() groups elements into a Map based on a classifier function.

Interview Summary (Part 2)

Operation	Purpose	Type
distinct()	Remove duplicates	Intermediate
limit()	First N elements	Intermediate
skip()	Ignore first N	Intermediate
count()	Count elements	Terminal
collect()	Create List/Set/Map	Terminal
groupingBy()	Group data	Collector (used with collect())

Overall Interview Questions (With Answers)

Q1. How does distinct() remove duplicates?

Answer: By using the equals() and hashCode() methods.

Q2. Is limit() intermediate or terminal?

Answer: Intermediate.

Q3. Is skip() intermediate?

Answer: Yes.

Q4. What does count() return?

Answer: A long.

Q5. Which operation is commonly used for pagination?

Answer: skip() together with limit().

Q6. Which terminal operation creates a List?

Answer: collect(Collectors.toList()).

Q7. Difference between toList() and toSet()?

Answer: toList() preserves duplicates; toSet() removes duplicates.

Q8. Can Collectors.toMap() throw an exception?

Answer: Yes. It throws IllegalStateException if duplicate keys exist and no merge function is supplied.

Q9. What does groupingBy() return?

Answer: A Map> by default.

Q10. Which Stream operation is similar to SQL GROUP BY?

Answer: Collectors.groupingBy().

Rapid Interview Revision

- distinct() → Removes duplicates using equals() and hashCode().
- limit() → Takes the first **N** elements.
- skip() → Ignores the first **N** elements.

- `count()` → Returns the number of elements as a long.
- `collect()` → Converts a stream into a List, Set, Map, or other result.
- `Collectors.toList()` → Creates a List.
- `Collectors.toSet()` → Creates a Set (duplicates removed).
- `Collectors.toMap()` → Creates a Map; duplicate keys require a merge function.
- `Collectors.groupingBy()` → Groups elements like SQL GROUP BY, returning a Map of grouped results.